Description

Objective

Design and build a robust, high-performance content delivery API that leverages modern HTTP caching standards and integrates seamlessly with a Content Delivery Network (CDN). The primary goal is to minimize latency for global users and reduce load on the origin server by maximizing cache hit rates at the edge. You will implement sophisticated caching strategies, including ETags for conditional requests, granular Cache-Control headers, and immutable content versioning.

This project will challenge you to think about API design from a performance and scalability perspective. You will gain practical experience in configuring CDNs, managing assets in cloud storage, and securing content while keeping it cacheable. The final service will be a production-grade system capable of delivering static and dynamic content efficiently and securely.

Core Requirements

Data Models

- `Asset`: Must store metadata for each piece of content, including a unique identifier, object storage key, filename, MIME type, size, a strong ETag, current version number, and timestamps.
- `AssetVersion`: Must track immutable versions of an asset, linking back to the parent asset and storing its specific object storage key.
- `AccessToken`: Must support temporary, secure access to private assets, containing a unique token, an expiration time, and a reference to the asset it grants access to.

API Endpoints & Functionality

- `POST /assets/upload`: Accepts a file upload, stores it in a cloud object storage service, creates an `Asset` record in the database, and returns the asset's metadata.
- `GET /assets/:id/download`: Serves the asset's content. This endpoint must support conditional `GET` requests using `ETag` and `If-None-Match` headers.
- `HEAD /assets/:id/download`: Allows clients to check for content modifications without downloading the body, responding with appropriate headers including the `ETag`.
- `POST /assets/:id/publish`: Creates a new, immutable `AssetVersion` for an existing asset, allowing content updates without breaking caches for old versions.
- `GET /assets/public/:version_id`: A public, highly-cacheable endpoint for serving versioned, immutable content directly from the CDN.
- `GET /assets/private/:token`: Provides access to private content using a valid, non-expired `AccessToken`.

HTTP Caching Requirements

- **ETag Generation:** The system must generate a strong ETag (e.g., an MD5 or SHA-1 hash of the file's content) for every asset upon upload or update.
- **Conditional Requests:** The download endpoints must correctly handle the `If-None-Match` request header. If the client's ETag matches the server's, the API must respond with a `304 Not Modified` status and an empty body.
- **Cache-Control Headers:** The API must set appropriate `Cache-Control` headers based on the content type:
  - Public, versioned assets: `public, max-age=31536000, immutable`
  - Public, mutable assets: `public, s-maxage=3600, max-age=60`
  - Private assets: `private, no-store, no-cache, must-revalidate`
- **Last-Modified Header:** The API should also include the `Last-Modified` header in responses for cache validation.

CDN and Storage Integration

- **Object Storage:** The service must use a cloud object storage provider (e.g., AWS S3, Google Cloud Storage) as the source for all assets.

- **Cache Invalidation:** The system must provide a mechanism to invalidate CDN caches when a mutable asset is updated, for instance, by calling the CDN's purge API.

Performance & Security

- **Cache Hit Ratio:** The final system, under simulated load, must be designed to achieve a CDN cache hit ratio of over 95% for public assets.
- **Origin Shielding:** The API's download endpoints should be protected to only allow traffic from the designated CDN, preventing direct access.
- **Secure Tokens:** The token generation mechanism for private assets must be cryptographically secure and tokens must have a short, configurable lifespan.

Implementation Guidelines

Technical Stack Suggestions

- **Language:** Consider statically-typed languages like Go, or dynamically-typed languages with mature ecosystems like Python or Node.js.
- **Web Framework:** Choose a high-performance framework like Gin (Go), FastAPI (Python), or Express/Fastify (Node.js).
- **Database:** A relational database like PostgreSQL is recommended for managing asset metadata and relationships.
- **Object Storage:** AWS S3, Google Cloud Storage, or a compatible alternative like MinIO for self-hosting.
- **CDN:** Cloudflare, AWS CloudFront, or Fastly are excellent choices with powerful caching and security features.

Architectural Design

- **Decouple Storage:** Strictly separate the API server from the file storage. The API should handle metadata and authorization, while generating signed URLs or proxying requests to the object store.
- **ETag Generation:** Generate ETags during the file upload process by hashing the file content. Store this ETag in your database alongside the asset metadata. This avoids re-calculating the hash on every request.
- **CDN Configuration:** Focus on setting up caching rules (or Page Rules/Workers in Cloudflare) that respect the `Cache-Control` headers sent by your origin API. Configure an "Origin Shield" or similar feature to protect your API from direct traffic.
- **Cache Invalidation Strategy:** For mutable assets, consider a programmatic approach. When an asset is updated, your API should trigger a purge request to the CDN's API. For versioned assets, invalidation is unnecessary; you simply point clients to the new version's URL.

Outcomes

- A fully functional API that can upload, version, and serve public and private assets from cloud storage.
- Correct implementation of HTTP caching, demonstrated by the API returning `304 Not Modified` responses for conditional requests with matching ETags.
- Appropriate `Cache-Control`, `ETag`, and `Last-Modified` headers are present and correctly configured on all content-serving responses.
- A CDN is successfully configured to cache public assets, achieving a cache hit rate of over 95% during testing.
- Private assets are inaccessible without a valid, short-lived access token.
- A robust content versioning system that allows for immutable caching of specific asset versions.
- A documented and functional cache invalidation process for mutable assets.
- A comprehensive test suite that verifies caching logic, endpoint functionality, and security controls.